

Portage du Bytestream Decoder sur Coyote

**Master of Science HES SO in
Engineering**

Profil/Orientation Computer Science

Date of publication: 03.06.2026

Produced by:

Author Abivarman Kandiah

Under the supervision of Andres Upegui Posada, Quentin Berthet



Table of Contents

Declaration of authenticity	3
Acknowledgments	4
Resume (English version)	5
Résumé (version française)	5
1. Context (Résumé Temporaire)	6
2. Introduction	6
3. Contexte technique	7
3.1. Accélération matérielle par FPGA pour le traitement de flux	7
3.2. Frameworks de développement hôte-FPGA	7
3.2.1. XRT	8
3.2.2. Coyote	8
3.3. Bytestream Decoder	8
4. Prise en main et évaluation de Coyote	9
4.1. Mise en place de l'environnement	9
4.2. Tests sur U55C — difficultés rencontrées	9
4.3. Tests sur V80 — difficultés rencontrées	10
5. Portage du Bytestream Decoder sur Coyote	11
5.1. Adaptation du wrapper hardware (Design 1)	11
5.2. Adaptation du code hôte	11
5.3. Design 2 — sortie 128 bits	12
6. Analyse des performances et optimisation	13
6.1. Méthodologie de mesure (ILA)	13
6.2. Design 1 — diagnostic du goulot	13
6.3. Design 2 — sortie 128 bits	13
6.4. Montée en fréquence à 400 MHz	13
6.5. Tableau comparatif et bilan	13
7. Discussion : Coyote face à XRT et perspectives smartNIC	13
7.1. Coyote vs XRT — bilan du portage	13
7.2. Pertinence pour des cas d'usage de type smartNIC	14
7.3. Perspectives d'amélioration	14
8. Conclusion	14
Bibliography	15
A Code — Wrapper vfpga_top.svh	17
B Script d'analyse ILA	17





Declaration of authenticity

I hereby affirm that this assignment is my own written work and that I have used no other sources or aids other than those indicated. All passages that have been quoted from publications or paraphrased from these sources are indicated as such. Additionally, any assistance from artificial intelligence tools has been appropriately acknowledged and documented.

Date : 03.06.2026

Name : Abivarman Kandiah



structured-mse-thesis.typ	To be finalized	
HES-SO / Domaine I&A / MSE / Travail de Master	03-06-2026 - Abivarman Kandiah	- 3 / 17 -



Acknowledgments

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.





Resume (English version)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Résumé (version française)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.





1. Context (Résumé Temporaire)

L'idée du projet était la prise en main de Coyote, et le test de celui-ci avec des algorithmes déjà implémenté sous XRT. Si possible, aussi confirmer le fonctionnement de celui-ci sur le FPGA V80 qui est nouvellement supporté par Coyote et que nous possédons mais qui était pas vraiment utilisable car ne supporte pas XRT.

J'ai commencé par faire fonctionner Coyote avec le premier design d'exemple sur la U55C. Durant celui-ci j'ai pu rencontré certains problèmes. certains ont été fixés, d'autres je pensais l'avoir fix mais en fait au final il n'est toujours pas stable. En effet, le design se programme bien et le driver semble bien se charger, mais lors de l'exécution de l'host, le programme bloque car n'arrive pas à exécuter le transfert avec le FPGA.

En parallèle, j'ai pu prendre en main en testant le programme qui a été fait par Upegui, qui est le bytestream decoder sous XRT qui est un projet dans le cadre du projet ATLAS. Après avoir testé le bon fonctionnement de celui-ci, j'ai pu commencé à porter le design et le code hôte pour une utilisation sous coyote.

Entre temps, la V80 était officiellement supporté, donc j'ai pu aussi commencé à tester le design d'exemple sous la V80. Avec la V80, j'avais rencontré un problème principal, ou je n'arrivais pas à charger correctement le driver. J'avais un soucis de manque de mémoire pour MSI-X. J'ai pu réglé le problème en ajoutant dans les bootargs, le `pci=realloc`.

Au final, la V80 marche assez bien sous coyote, j'arrive à reproduire correctement à chaque fois contrairement à la U55C.

Le travail du portage côté design est la réécriture du Wrapper, et le fait que coyote a un bus AXI-S de 512 bits, tandis que le design utilise des 32 bits.

2. Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam ipsum accusantium delectus officiis ut ut. Ut enim ad voluptatem dolorum aperiam doloribus nostrum voluptate reiciendis et eum. Ut itaque id est autem. Ut enim aut voluptatibus tempore, atque ut vero sunt non culpa. Voluptatem et voluptatibus nostrum. Ut enim aut voluptatibus tempore, atque ut vero sunt non culpa. Voluptatem et voluptatibus nostrum. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.



3. Contexte technique

3.1. Accélération matérielle par FPGA pour le traitement de flux

Les FPGAs sont des matériels utiles à des applications nécessitant des traitements de données à très faible latence, avec des exigences de débit élevées, et une flexibilité d'architecture

Contrairement aux CPUs et aux GPUs, dont l'architecture est figée et le parallélisme borné par le nombre de cœurs ou de threads, un FPGA permet de décrire une architecture sur mesure, dans laquelle chaque étage de traitement s'exécute en parallèle des autres. Cette propriété rend les FPGAs particulièrement adaptés au traitement de flux de données en continu.

Deux cartes ont été utilisées dans le cadre de ce projet. La première, l'Alveo U55C, est une carte d'accélération basée sur un FPGA UltraScale+; La seconde, la V80, est plus récente : elle repose sur l'architecture Versal (NoC matériel intégré, AI Engines), expose une interface PCIe Gen5 x8 et embarque également de la HBM [1].

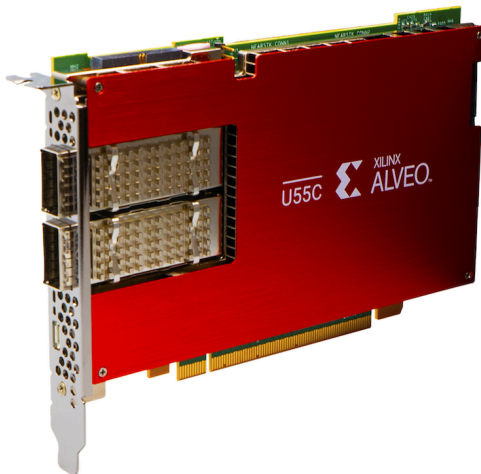


Figure 1: Xilinx Alveo U55C, une carte d'accélération basée sur un FPGA UltraScale+.

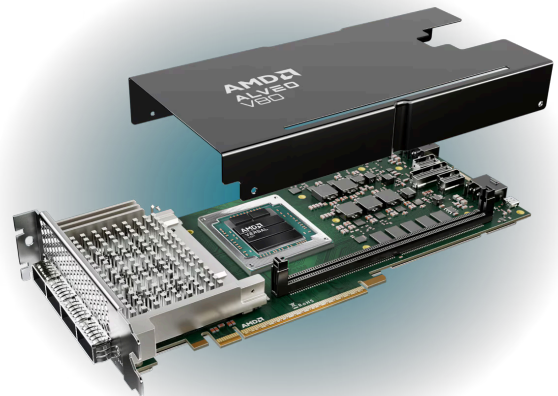


Figure 2: Xilinx Versal V80, une carte d'accélération basée sur un FPGA Versal.

3.2. Frameworks de développement hôte-FPGA

Pour faciliter le développement et le déploiement d'applications sur FPGA, ils existent des frameworks nous fournissant un shell qui est une sorte de système d'exploitation pour FPGA, qui gère les aspects génériques de la communication avec l'hôte (PCIe, DMA, gestion mémoire, etc.) laissant à l'utilisateur la liberté de se concentrer sur la logique applicative. Le shell va généralement permettre à l'utilisateur de charger dynamiquement une application logique sur le FPGA, sans avoir à reprogrammer le FPGA en entier.

Ces frameworks fournissent également une API pour l'hôte, permettant de faciliter les interactions avec le FPGA tel que le chargement de la logique applicative et le transfert de données.

3.2.1. XRT

XRT est le framework officiel d'AMD/Xilinx qui va cibler principalement leurs cartes FPGA [2]. La programmation de la couche logique applicative repose sur des kernels généralement décrits en HLS. La transition des données entre l'hôte et le FPGA suit un schéma comme celui-ci :

XRT bénéficie d'un écosystème mature, intégré à la toolchain Vitis, mais malheureusement ne supporte pas les cartes plus récentes comme la V80.

3.2.2. Coyote

Coyote est un framework open source développé à l'ETH Zürich [3]. La logique applicative y prend la forme d'un ou plusieurs **vFPGAs** (*virtual FPGAs*), qui dialoguent avec le shell via des interfaces AXI-Stream de 512 bits. Plusieurs vFPGAs peuvent coexister dans le même shell, ce qui permet d'isoler plusieurs applications sur le même FPGA.

Le top-level du vFPGA s'écrit en SystemVerilog, mais la logique applicative en dessous peut être décrite en VHDL, Verilog, SystemVerilog, HLS, ou encore SpinalHDL.

Coyote propose également plusieurs modes de transfert de données entre l'hôte et le FPGA, comme illustré sur le schéma suivant :

Le mode de transfert qui va nous intéresser en particulier c'est le mode "Local Read/Write/Transfer" qui va permettre de streamer les données directement entre la mémoire de l'hôte et le vFPGA, sans passer par une mémoire intermédiaire sur la carte. Enfin, Coyote supporte depuis récemment la V80, ce qui en fait à ce jour l'une des rares options viables pour exploiter cette carte.

3.3. Bytestream Decoder

L'algorithme retenu pour tester Coyote sur un cas concret est le **Bytestream Decoder**, développé pour l'expérience ATLAS au CERN [4]. Il sert dans la chaîne de lecture du calorimètre à argon liquide (LAr), en tête du pipeline d'accélération du **Topo-automaton clustering**.

Le flux d'entrée contient, sous forme brute, les données mesurées pour chaque cellule du calorimètre : gain, énergie, temps et qualité. Le décodeur va lire ce flux et reconstruire à la volée une structure exploitable côté logiciel, appelée **CaloCell container**.

Cette implémentation est intéressante car le décodeur existe déjà sous la forme d'un design VHDL validé sous XRT sur des cartes Alveo [5], ce qui va nous donner un point de comparaison direct avec Coyote.

Le pipeline prend en entrée deux flux : le ByteStream brut (700 KB) et des données de correction (7 KB).

En interne, le pipeline travaille sur un bus de 32 bits. Le module expose donc deux flux AXI-Stream 32 bits en entrée, et produit en sortie quatre mots de 32 bits par cellule : Gain+ID, Energy, Time, Quality. Cette largeur de 32 bits, héritée de l'implémentation XRT d'origine, va jouer un rôle important dans l'analyse de performance présentée plus loin.

4. Prise en main et évaluation de Coyote

4.1. Mise en place de l'environnement

Le flot de développement avec Coyote se base sur CMake et Vivado 2024.2 (pour la V80). Le design hardware se build via CMake en précisant la carte cible avec `-DFDEV_NAME=u55c` ou `-DFDEV_NAME=v80`.

Le driver noyau se compile à part. Pour la V80, il faut préciser `TARGET_PLATFORM=versal` au `make`, sinon le driver généré ne correspond pas à la carte.

Un cycle de déploiement complet ressemble donc à ça :

1. build du bitstream
2. programmation du FPGA via Vivado
3. remove et rescan du bus PCIe (sinon le device n'est pas redétection après reprogrammation)
4. chargement du driver Coyote
5. exécution du programme hôte

Pour valider la chaîne, j'ai commencé par l'exemple `01_hello_world` fourni avec Coyote, qui réalise un simple transfert hôte ↔ FPGA. C'est sur cet exemple que la majorité des problèmes de mise en route ont été rencontrés.

4.2. Tests sur U55C — difficultés rencontrées

Deux problèmes principaux sont apparus lors de la mise en route sur l'Alveo U55C.

Device PCIe non détecté après reprogrammation du FPGA. Quand on reprogramme le FPGA, le device PCIe n'est plus directement utilisable côté hôte, ce qui fait planter le programme de test avec une erreur du type `cThread instance could not be obtained`. Au départ, je ne faisais pas la bonne manipulation pour réinitialiser le bus PCIe. Il faut en réalité forcer un **remove** du device, puis un **rescan** du bus, avant de recharger le driver :

```
sudo sh -c "echo 1 > /sys/bus/pci/devices/0000:21:00.0/remove"
sudo sh -c "echo 1 > /sys/bus/pci/rescan"
```

Programme hôte bloquant pendant le transfert. Le design est programmé, le driver se charge sans erreur, mais le programme hôte reste bloqué pendant le transfert. Initialement, on pensait que le problème venait de l'IOMMU qui n'était pas en mode passthrough. Ajouter `amd_iommu=on iommu=pt` dans les bootargs GRUB semblait fixer le souci au début, mais au final non : le comportement est resté instable.

Dans certains cas, même avec le bitstream et le driver chargés correctement, le programme hôte se bloque toujours sur le transfert. Une analyse avec l'ILA montre que le signal `tvalid` de l'interface réceptrice ne passe jamais à 1, donc le transfert ne démarre pas du tout côté FPGA. La cause exacte n'a pas pu être identifiée dans le temps imparti.

Au final, la U55C n'est pas fiable pour faire des mesures reproductibles, ce qui nous a poussé à utiliser la V80 comme plateforme principale.



4.3. Tests sur V80 — difficultés rencontrées

La V80 venait d'être officiellement supportée par Coyote au moment du projet, donc valider ce support faisait partie des objectifs.

Le problème principal rencontré était au chargement du driver, avec une erreur liée au MSI-X. La carte expose un grand nombre de vecteurs d'interruption, et le BIOS ne réserve pas assez de mémoire PCI pour les allouer. Le fix consiste à ajouter `pci=realloc=on` dans les bootargs GRUB pour forcer le kernel à réallouer les ressources PCI au boot.

Une fois ce problème réglé, la V80 fonctionne de manière stable et reproductible. À noter cependant qu'à chaque reprogrammation de la V80 via Vivado, il peut être nécessaire de redémarrer la machine hôte pour forcer une réallocation propre du PCI au boot, sinon on peut rencontrer à nouveau cette erreur.

C'est cette carte qui a servi de plateforme pour toutes les mesures de performance présentées plus loin.



5. Portage du Bytestream Decoder sur Coyote

Le portage du Bytestream Decoder de XRT vers Coyote consiste à refaire le wrapper au format attendu par Coyote, et réécrire le code hôte autour de l'API Coyote. Le coeur VHDL du décodeur, lui, reste identique à la version XRT d'origine.

5.1. Adaptation du wrapper hardware (Design 1)

Le shell Coyote expose un bus AXI-Stream de 512 bits côté vFPGA, alors que le Bytestream Decoder travaille en interne sur des mots de 32 bits. Le rôle du wrapper, écrit en SystemVerilog dans `vfpga_top.svh`, est donc d'instancier le module VHDL et de connecter ses ports 32 bits aux bits de poids faible de chaque interface AXI-Stream.

Concrètement :

- En entrée, seuls les bits `[31:0]` des flux `axis_host_recv[0]` (données brutes) et `axis_host_recv[1]` (corrections initiales) sont connectés au décodeur.
- En sortie, les bits `[31:0]` du flux `axis_host_send[0]` portent un mot de sortie par cycle.
- Les 480 bits restants de chaque beat ne sont pas utilisés. Le bus est donc sous-utilisé d'un facteur 16, ce qui est volontaire pour ce premier design.

Le wrapper instancie aussi une ILA décrit via le script `init_ip.tcl`, avec des sondes sur les signaux axi-stream des interfaces d'entrée et de sortie. Cela va permettre de mesurer les duty cycles côté FPGA et d'identifier les goulots dans l'analyse de performance présentée plus loin.

5.2. Adaptation du code hôte

Le code hôte a été entièrement réécrit autour de l'API Coyote, à la place du modèle `xclbin` + buffers de XRT.

Le programme commence par charger les deux fichiers d'entrée :

- `Uncompressed_data_LAR_only.raw` : 174 736 mots de 32 bits (données brutes du bytestream)
- `initial_correction_values.bin` : 1 833 mots de 32 bits (valeurs de correction)

Coyote travaillant sur des blocs de 512 bits, chaque mot 32 bits est placé dans les bits de poids faible d'un bloc de 64 octets, les 60 octets restants étant remplis de zéros.

Les données sont ensuite envoyées au FPGA via deux flux distincts :

- Stream 0 : les données brutes du bytestream
- Stream 1 : les valeurs de correction, envoyées une seule fois en début d'exécution

Le transfert principal est déclenché par deux appels à `coyote_thread.invoke()` : un `LOCAL_READ` pour pousser les données brutes vers le FPGA, et un `LOCAL_WRITE` pour récupérer la sortie. Le programme attend ensuite la complétion des deux opérations.

Côté sortie, chaque CaloCell occupe quatre blocs de 512 bits successifs, à raison d'un mot 32 bits par bloc : Gain+ID, Energy, Time, puis Quality. L'unpacking parcourt donc le buffer de sortie quatre blocs à la fois pour reconstruire chaque cellule.



5.3. Design 2 — sortie 128 bits

La sortie du décodeur, une cellule **CaloCell**, est constituée de 4 mots de 32 bits. Dans le Design 1, ces 4 mots sont émis un par un par l' `Output_Merger` , ce qui demande 4 blocs de 512 bits sur le bus AXI-Stream pour transférer en réalité seulement 128 bits utiles.

L'idée du Design 2 est de les regrouper en un seul mot de 128 bits, et de les envoyer directement dans un seul bloc de 512 bits. On passe ainsi de 4 blocs à 1 bloc par cellule, soit un facteur $\times 4$ sur la bande passante utile en sortie.

À noter que la variante 128 bits de l' `Output_Merger` était déjà présente dans le code source VHDL, mais c'est la variante 32 bits (`Output_Merger_32`) qui était utilisée pour le portage sous XRT. Les deux versions cohabitent donc dans `Byte_Stream_Decoder_parallel.vhd` , et il a suffi de swapper l'instanciation pour réactiver la variante 128 bits.

Côté wrapper, le changement se résume à connecter les bits `[127:0]` du flux de sortie au lieu des bits `[31:0]` . Les 384 bits restants du bloc ne sont toujours pas utilisés, mais le ratio passe de 1/16 à 4/16.

Côté hôte, l'unpacking est ajusté pour lire une cellule complète par bloc, en extrayant les 4 champs (Gain+ID, Energy, Time, Quality) dans les 16 octets de poids faible.

Le reste de la chaîne reste strictement identique au Design 1.



6. Analyse des performances et optimisation

6.1. Méthodologie de mesure (ILA)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.2. Design 1 – diagnostic du goulot

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.3. Design 2 – sortie 128 bits

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

6.4. Montée en fréquence à 400 MHz

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.5. Tableau comparatif et bilan

Design	Fréquence	Sortie	Temps transfert	tready entrée
Design 1	250 MHz	32 bits	3.17 ms	20.9 %
Design 2	250 MHz	128 bits	0.83 ms	88.7 %
Design 2	400 MHz	128 bits	0.82 ms	90.0 %

Table 1: Comparaison des performances des trois configurations testées sur V80.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

7. Discussion : Coyote face à XRT et perspectives smartNIC

7.1. Coyote vs XRT – bilan du portage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod



idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.2. Pertinence pour des cas d'usage de type smartNIC

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad eaue doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.3. Perspectives d'amélioration

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad eaue doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

8. Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad eaue doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.



Bibliography

- [1] AMD, “AMD Alveo V80 Compute Accelerator Card – Product Brief.” 2024.
- [2] AMD/Xilinx, “XRT: Xilinx Runtime Library Documentation.” 2024.
- [3] D. Korolija, T. Roscoe, and G. Alonso, “Do OS Abstractions Make Sense on FPGAs?,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/roscoe>
- [4] ATLAS LAr Collaboration, “The ATLAS Liquid Argon Calorimeter: Construction, Integration, Commissioning and Performance,” *Journal of Instrumentation*, vol. 5, p. P9003, 2010.
- [5] A. Upegui Posada, “Bytestream Decoder – design VHDL pour ATLAS LAr sur Alveo U55C.” 2024.
- [6] AMD/Xilinx, “Alveo U55C High Performance Compute Card – Product Brief.” 2022.
- [7] D. Firestone and others, “Azure Accelerated Networking: SmartNICs in the Public Cloud,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2018.



Appendices

A Code — Wrapper vfpga_top.svh	17
B Script d'analyse ILA	17





A Code — Wrapper vfpga_top.svh

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

B Script d'analyse ILA

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

